

CHAPTER 5

SYSTEM DESIGN

The system design phase establishes the structural and functional framework of the Smart Student Campus Recruitment System and defines how the complete solution will be organized and implemented. It determines the architecture of the platform, the various functional modules, and the interactions among the components that together ensure a reliable and efficient recruitment process. Similar to the requirement analysis stage, the system design undergoes a thorough review by software quality assurance personnel to verify that every element of the design aligns with project objectives, user expectations, and technical feasibility. The documentation associated with the design review is maintained within the organizational standard repository to ensure traceability, consistency, and continuous improvement.

A systematic approach to system design is essential for creating a coherent, scalable, and secure digital recruitment platform. To achieve this, both Top-Down and Bottom-Up methodologies are applied so that each module is fully integrated with the overall platform while also functioning independently without dependency conflicts. The system's design models incorporate every variable associated with users, data flow, authentication, and decision-making to ensure seamless communication between students, companies, and placement officers. Standard modelling languages are used to represent the flow of information and relationships between system components, ensuring that the design follows a uniform structure governed by well-defined rules and semantics.

The Smart Student Campus Recruitment System design can be represented using graphical modelling diagrams such as Data Flow Diagrams, Use Case Diagrams, Class Diagrams, and Sequence Diagrams, which collectively illustrate system behaviour, actor interactions, internal processing, and storage mechanisms. These design representations allow software engineers and evaluators to clearly understand how the system is expected to function before implementation begins. By maintaining modelling accuracy and consistency throughout the design phase, the project ensures a strong blueprint that supports secure, scalable, and effective deployment of the recruitment system.

5.1 Input Design

Input design plays an extremely important role in the development of the Smart Student Campus Recruitment System because it determines how users interact with the software and how the system collects and validates the information. A well-designed input structure ensures that the right

data is captured with accuracy, consistency, and reliability. In this recruitment system, inputs are collected mainly from three categories of users: students, placement officers, and company recruiters. Each category interacts with the system using role-based access, which means that the input fields are tailored specifically to the tasks and responsibilities of the user.

Students provide input during registration, where they enter personal information, including name, hall ticket number, email address, academic details, phone number, and password. After logging in, students continue to enter additional information, especially while creating and updating their resumes. This includes sections such as education history, internships, certifications, projects, skills, personal strengths, and goals. Students also input data when applying for job posts by responding to eligibility fields and submitting their resumes. Because student information is crucial for recruitment actions, the input process uses form validation, mandatory fields, character length checks, and authentication to ensure accuracy and prevent invalid records.

Placement Officers interact with the system using more administrative control for data input. They register companies, update student details whenever necessary, upload or modify job postings, and manage eligibility criteria. Placement officers also add shortlisting results and placement updates, which serve as key decision-making information for students and companies. Because this user category deals with sensitive recruitment data, their input process supports authority validation, database integrity checks, and secure login authorization.

Companies provide input primarily when they register and publish job opportunities. During registration, they input company profile information such as organization name, address, website, HR contact details, and an official email address. Later, when posting jobs, they enter fields such as job title, salary package, eligibility criteria, required CGPA cutoff, job description, and recruitment deadlines. Recruiters also update the results by selecting shortlisted candidates. The input design ensures that all fields associated with job creation and student shortlisting are structured properly, so that placement processing becomes systematic and error-free.

Overall, the input design in the recruitment system emphasizes usability, error-free data collection, and streamlined communication among all users. It aims to reduce data redundancy, minimize user workload, and ensure that the information stored is highly accurate and securely processed inside the system.

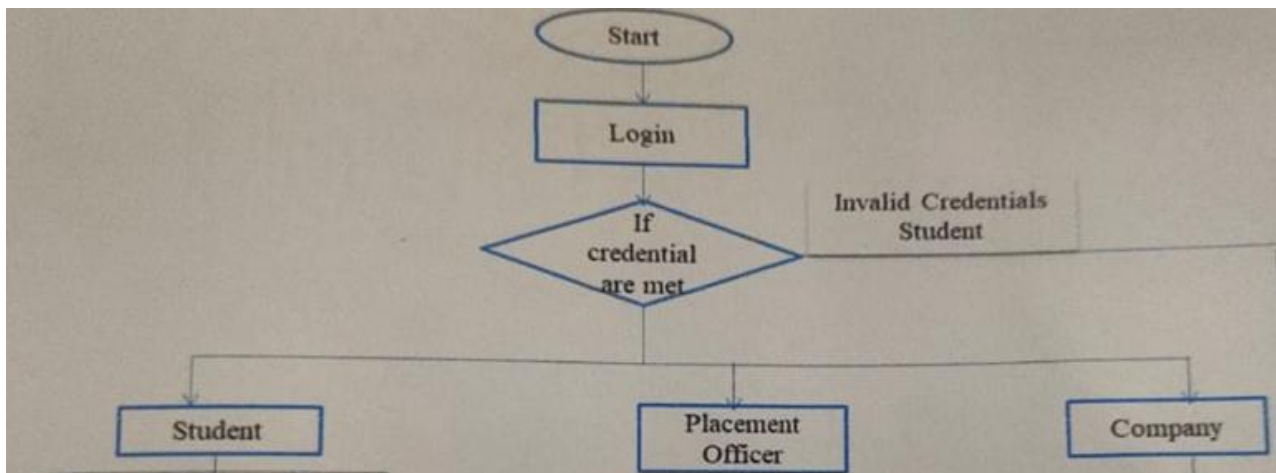


Fig 5.1 :- Input design

The design stage of the Smart Student Campus Recruitment System begins by converting all functional and non-functional requirements from the approved SRS into detailed design components that guide system development, as shown in Figure 5.1. Through discussions with stakeholders and prototype evaluations, the requirements were translated into functional hierarchy charts, screen layout diagrams, business rule tables, and business process diagrams. Pseudo-code and an Entity-Relationship Diagram with a data dictionary further define the internal system logic and database structure for students, companies and placement activities. Together, these design elements provide programmers with a clear and complete blueprint for building the system securely, efficiently, and with minimal ambiguity.

5.2 Output design

The design of output is the most important task of any system. During output design, developers identify the type of outputs needed, and consider the necessary output controls and prototype report layouts.

- The system generates meaningful outputs through dashboards, reports, notifications, and downloadable files, ensuring every stakeholder receives clear and actionable information for decision-making during the recruitment process. To develop the output design that meets the end users requirements.
- A 3-tier system architecture is followed, consisting of a Presentation Layer, Application Layer, and Data Layer, each handling specific responsibilities to improve performance and

reliability.

- The Presentation Layer delivers role-based access through secure login, allowing students, companies, and placement officers to view only the information relevant to their roles, such as eligible jobs, shortlisted candidates, and placement statistics.
- The Application Layer handles business logic, including eligibility checking, validation, application processing, result updates, resume access, alerts, and report generation, ensuring real-time communication and automation.
- The Data Layer securely stores student profiles, resumes, company details, job postings, and application statuses in a structured database, guaranteeing scalability, security, transparency, and seamless coordination across the campus recruitment process.

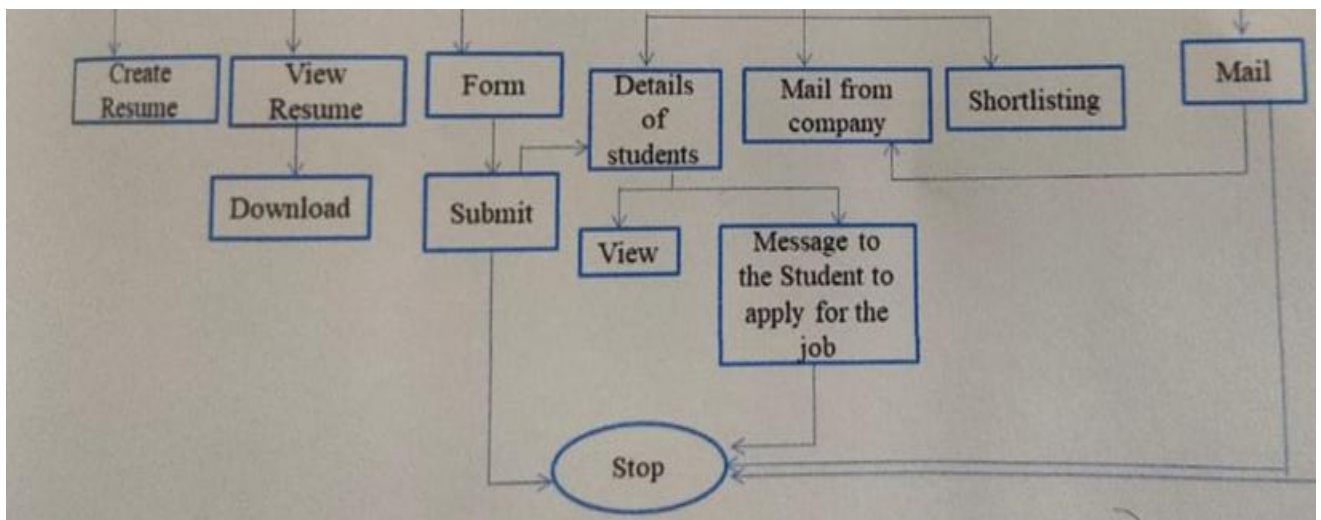


Fig 5.2 :- Output design

The development stage of the Smart Student Campus Recruitment System takes the design specifications finalized and documented during the system design phase as its primary input. For each design element, one or more software artifacts were developed and implemented to convert the design into a functional platform, as illustrated in Figure 5.2. These software artifacts include interactive menus for navigation, registration and login dialogs, student and company data management forms, resume upload and download interfaces, job posting modules, application tracking panels, administrative reporting formats, and automated procedures for eligibility validation, shortlisting, and notification triggers. For every related set of software artifacts, appropriate test cases are prepared to ensure their functionality, accuracy, and reliability before they are integrated into the system. Additionally, an online assistance/help section was incorporated to guide students, placement officers, and company recruiters in navigating and interacting with various features of the software. Through

this structured development approach, the system gradually evolved into a fully operational and user-ready recruitment platform.

5.3 System Architecture

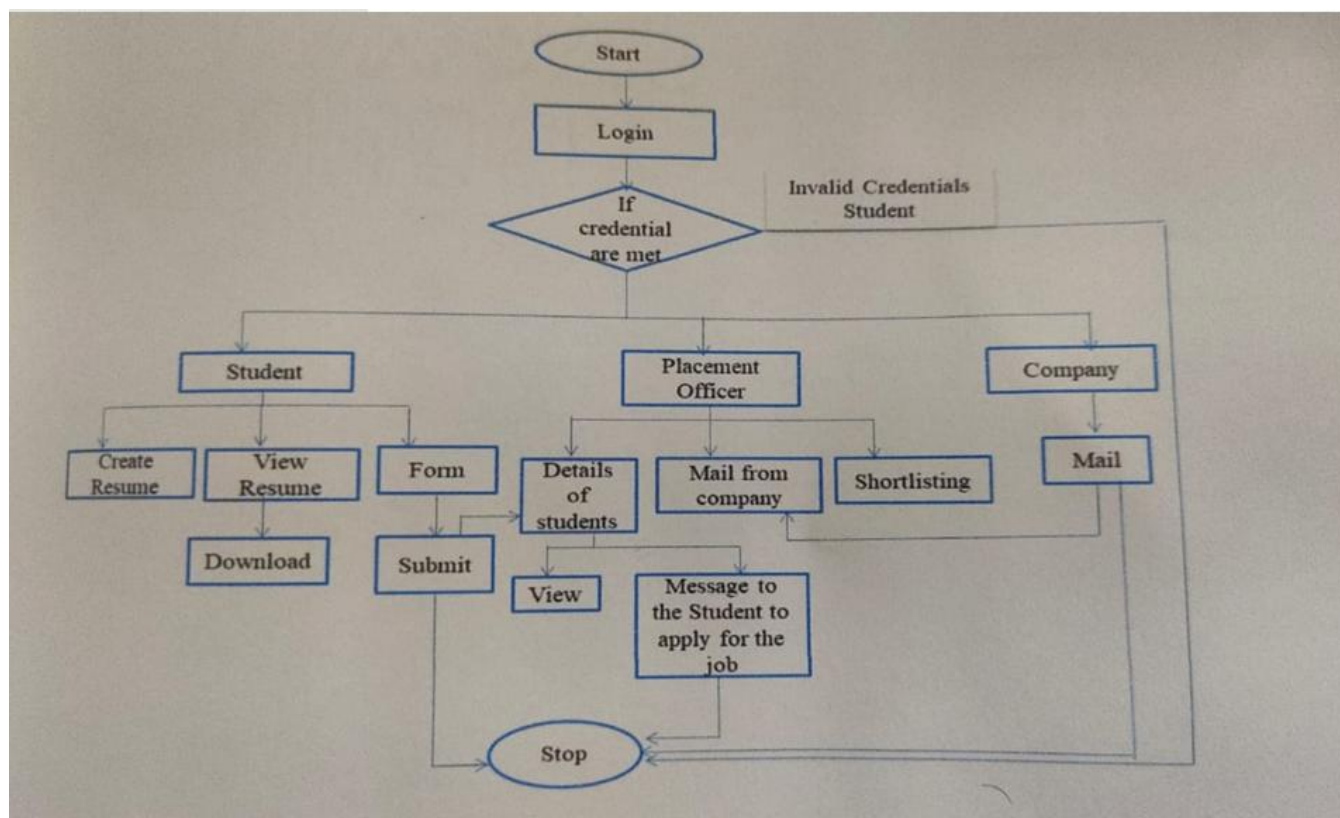


Fig 5.3 :- System architecture

The architecture of the Smart Student Campus Recruitment System follows a three-tier web-based model consisting of a presentation, application, and data layer, as illustrated in Figure 5.3. The presentation layer includes the browser interface used by students, placement officers, and recruiters, providing role-specific access to features such as job postings, application submissions, resume management and administrative data monitoring. The application layer functions as the logical core of the system, processing user requests, validating eligibility criteria, generating placement reports, and controlling access based on the roles and permissions assigned to each user category. The data layer stores information in a secure relational database that maintains structured and normalized records of students, companies, job postings, applications and placement outcomes. This layered architecture ensures centralized access, improved scalability, faster performance, and enhanced security, while allowing each layer to be updated or expanded independently without affecting the overall functioning of the recruitment platform.

5.4 Data flow diagram

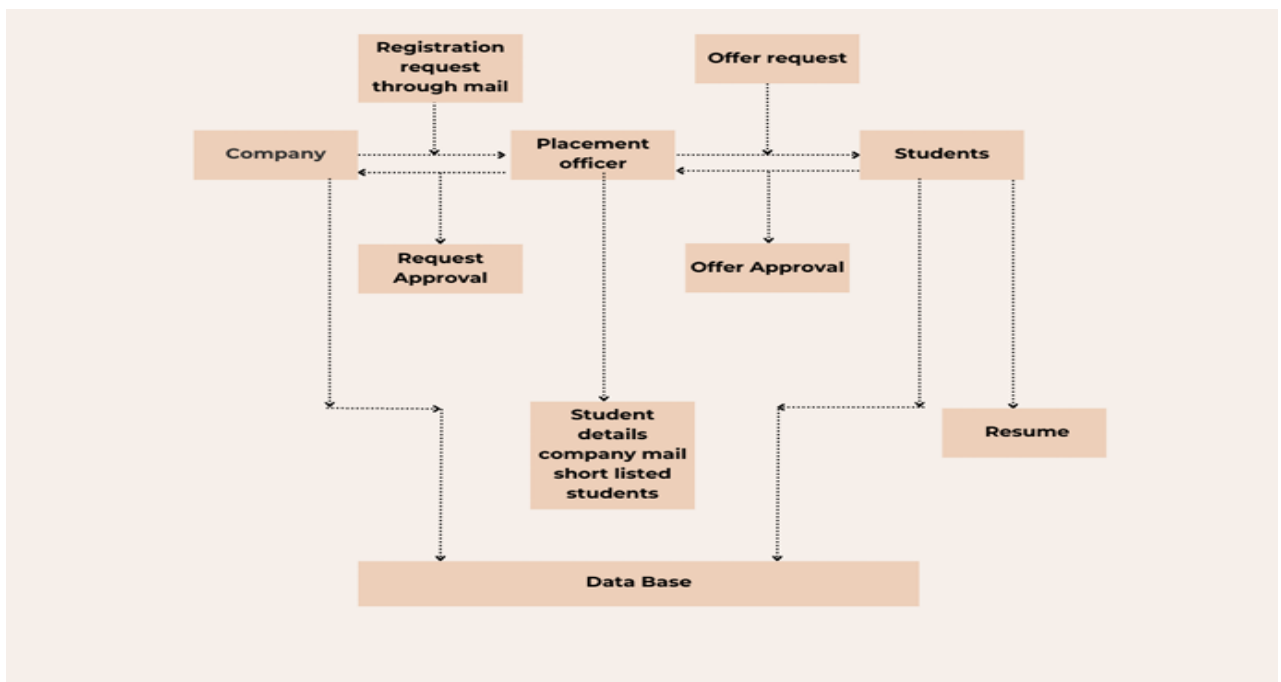


Fig 5.4 :- Data flow diagram

The data flow within the Smart Student Campus Recruitment System begins when a company expresses its intent to participate in campus placement by submitting a registration or job offer request to the placement officer, as shown in Figure 5.4. The placement officer reviews the request and approves it after validating its authenticity, after which the job posting becomes immediately visible to students who satisfy the specified eligibility criteria. Students then respond to job postings by submitting their resumes and application details through the system interface. All application submissions are securely stored in the database and are accessible to the placement officer for evaluation. The placement officer proceeds with the shortlisting process by comparing the academic background, skills, and resume content of applicants against company requirements. Once shortlisted, the system forwards the details of eligible students to the respective companies for further consideration. The company then reviews the submitted student profiles and resumes and may send interview invitations or recruitment outcomes back to the system through the placement officers. Throughout this workflow, the centralized database is continuously updated, ensuring the transparency, traceability, and synchronization of every recruitment decision. This digital flow of information eliminates manual paperwork, prevents miscommunication, reduces delays, and contributes to the creation of a structured, fast, and automated campus recruitment environment.

5.5 Sequence diagram

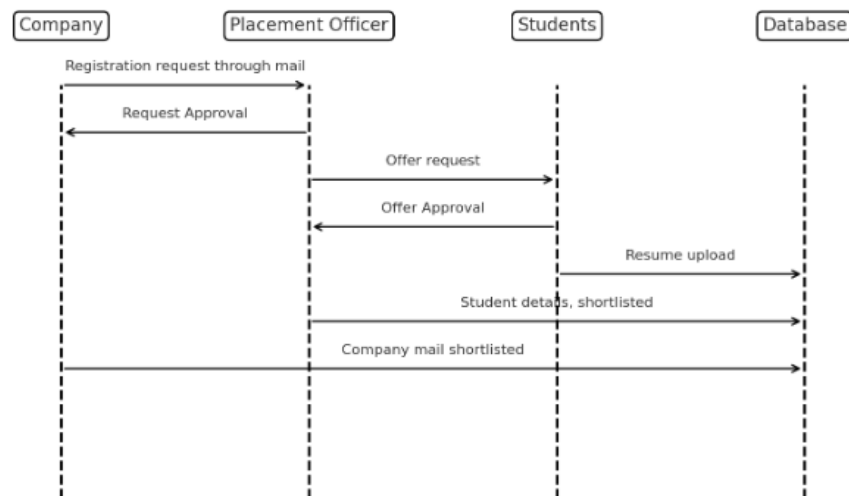


Fig 5.5 :- Sequence diagram

The sequence of operations in the Smart Student Campus Recruitment System begins when a student attempts to log in, as shown in Figure 5.5. If the credentials entered are valid, the system authenticates the student and grants access to a personalized dashboard, where the student can browse and view eligible job postings. After selecting a suitable job, the student submits an application, which is processed by the system and securely stored in a database. The placement officer later retrieves the complete list of applicants for that job posting and performs the shortlisting procedure based on eligibility criteria, academic performance, skill set, and resume suitability. Once the shortlisted candidates are finalized, the placement officer triggers automated notifications to both the students and the concerned company. In contrast, if the login credentials are invalid, the system immediately displays an alert message, and access is denied to ensure security. This sequence of interactions maintains logical communication between system components and users, reflecting an efficient and seamless flow throughout the recruitment cycle.

5.6 Class diagram

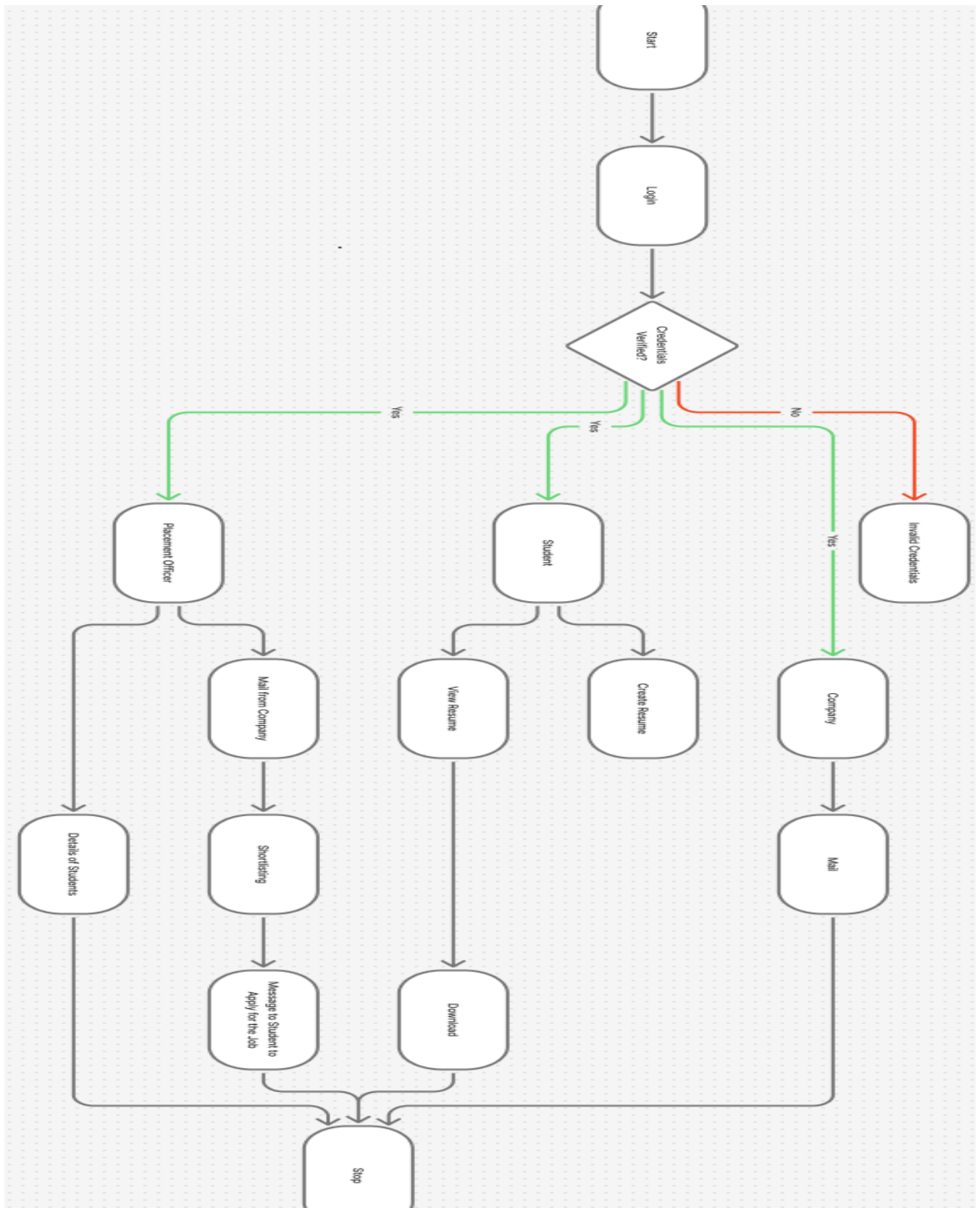


Fig 5.6 :- Class diagram

The class diagram of the Smart Student Campus Recruitment System represents the main entities of the software and the relationships that connect them, as shown in Figure 5.6. The Student class contains personal and academic attributes such as student ID, name, email, CGPA, and resume, along with methods responsible for registration, login, resume management, job application submission, and resume downloading. The Placement Officer class stores identification and contact details and provides methods that enable officers to manage student data, manage company profiles, update placement results, and generate analytical reports. The Company class represents recruiters and stores the company ID, organization name, job role, eligibility criteria, and HR contact information, along with functions to create job postings, view applicants, and download student resumes. To connect students and companies, the Job Application class is used to record application details such as application ID, student ID, company ID, and current application status, with methods that allow updating of shortlisting outcomes. The relationships among these classes form the backbone of the recruitment workflow, where a student can apply to multiple job postings, a company can receive applications from multiple students, and the placement officer supervises all recruitment interactions. The well-structured class design ensures data integrity, secure communication between modules, and efficient execution of all placement operations.

5.7 Use Case diagram

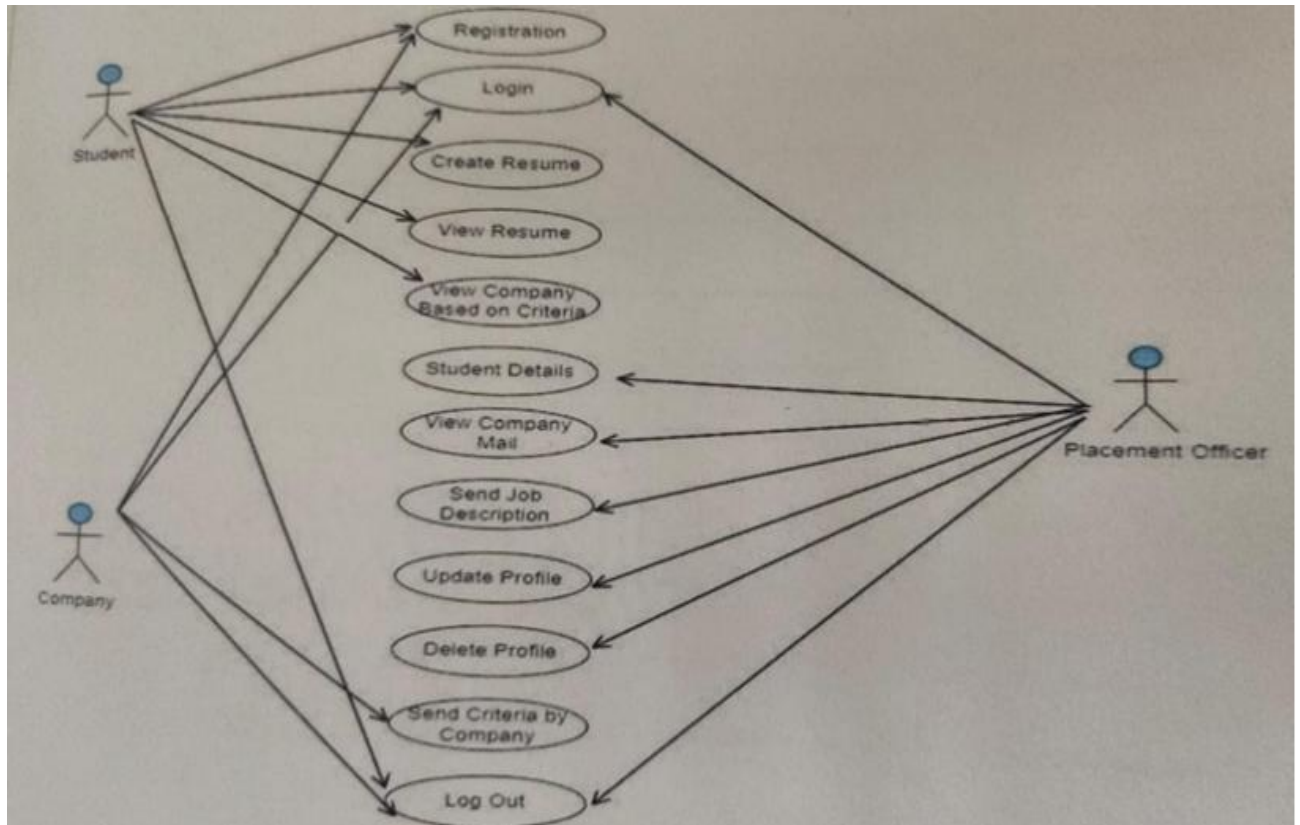


Fig 5.7 :- Use case diagram

The use case diagram of the Smart Student Campus Recruitment System identifies three main actors—students, placement officers, and companies—each interacting with the platform through functions defined by their respective roles, as illustrated in Figure 5.7. Students perform operations such as creating an account, logging in, building and viewing resumes, browsing eligible job postings, submitting applications, and logging out. Placement officers act as system administrators and interact with all modules by maintaining student records, registering companies, approving or modifying job postings, shortlisting candidates, and generating placement-related reports. Companies use the system to register, post job openings, review eligible applicants, download resumes, and communicate recruitment decisions. The use-case framework ensures that every actor has controlled and role-specific access to the platform according to their level of authority, thereby reinforcing role-based security and preventing unauthorized access to restricted information. By dividing the system into permission-based services, the use case model enhances usability, data confidentiality, and smooth execution of the overall recruitment workflow.

CHAPTER 6

IMPLEMENTATION

6.1 Language selection

The language selection for the Smart Student Campus Recruitment System was based on performance, scalability, platform compatibility, ease of development, and community support. Because the system is web-based and multi-user in nature, a combination of front-end and back-end technologies was chosen to ensure an interactive user interface and secure database communication.

6.1.1 Introduction:

The system uses HTML, CSS, and JavaScript for the front end to provide a responsive and user-friendly graphical interface that students, companies, and placement officers can access through web browsers. PHP is used as the server-side scripting language because of its reliable support for web application logic, fast execution capability, and easy integration with relational databases. MySQL was used as the database language because of its structured schema support, high execution speed, and security for storing academic, company, and application data. Collectively, these languages support modular development, efficient deployment, and seamless communication across different system modules.

6.1.2 Features:

The selected languages offer several features that make them suitable for this application:

- **HTML** enables structured page creation and content formatting for user dashboards and forms.
- **CSS** provides styling, layout control and responsive design to support access through multiple devices.
- **JavaScript** allows interactive elements such as live notifications, validation and dynamic content updates.
- **PHP** supports server-side computations, business logic handling, role-based authentication
- **MySQL** ensures structured data storage, fast execution of queries and secure handling of student, company and placement data with relational constraints.

6.1.3 Why these languages are preferred?

The combination of HTML, CSS, and JavaScript provides a highly flexible user interface that can be easily rendered on any device or operating system. PHP is preferred because it is scalable, lightweight, and widely used for dynamic web applications, making it ideal for implementing automated recruitment workflows. MySQL is favored for its robustness, strong ACID compliance, support for large

datasets, and ability to prevent data redundancy through normalization. All these languages are supported by large developer communities, extensive documentation, and powerful libraries, enabling faster development and troubleshooting. Together, they allow the creation of secure, responsive, and easily maintainable web applications.

6.1.4 Development Platform (XAMPP)

The system was developed using XAMPP, which integrates Apache web server, PHP interpreter and MySQL database in a single environment. It simplifies deployment, allows rapid prototyping and makes database connectivity seamless during development and testing phases.

6.1.5 Browser Execution Environment

Because the system is web-based, it can be executed using standard web browsers. Any device with modern browsers, such as Google Chrome, Mozilla Firefox, or Microsoft Edge, can access the application, ensuring cross-platform usability without the need for installation.

6.2 Platform Selection

The platform selection for the Smart Student Campus Recruitment System was influenced by the need for multi-user access, browser compatibility, ease of deployment, scalability, and security. Because the system is intended to be accessed by students, companies, and placement officers from different locations, a web-based platform was chosen to eliminate installation requirements and enable seamless access through any device with an Internet browser.

6.2.1 Development and Execution Platform

The system was developed and deployed locally using **XAMPP**, which provides an integrated environment combining the **Apache web server, PHP execution engine, and MySQL database server**. XAMPP enables smooth development and testing by simulating real-time web deployments on local machines. It supports reliable communication between PHP and the MySQL database, which is essential for authentication, job posting, application processing, and report generation.

6.2.2 Operating System and Hardware Support

The platform is compatible with multiple operating systems, including **Windows, Linux, and macOS**, ensuring flexibility for institutional use. Because the system is lightweight, it functions efficiently on standard hardware configurations used in computer labs or placement cells without requiring specialized equipment.

6.2.3 Browser Compatibility

The application is fully accessible using modern web browsers, such as **Google Chrome, Mozilla Firefox, and Microsoft Edge**. The responsive design ensures that the system operates smoothly on devices ranging from desktops and laptops to mobile phones and tablets, allowing students and recruiters to access services anytime and anywhere.

6.2.4 Advantages of the Selected Platform

The chosen platform offers several benefits, including centralized data management, simple deployment, scalability for handling increased users, easy migration to cloud hosting in the future, and reduced maintenance overhead. By adopting a web-based platform powered by XAMPP, the system provides high accessibility, smooth performance, and reliable security for the entire recruitment process.

6.3 Pseudocode

Pseudo Code is an informal high-level description of the operating principal of a computer program or other algorithm.

//Pseudo code for Student

- 1: If a login credential is equal to student entered credentials
- 2: If matched then go to student portal else display error message
- 3: end if
- 4: create resume and view resume by using user id
- 5: send details to placement officer using form
- 6: view company details based on the credentials
- 7: end if

//Pseudo code for Placement Officer

- 1: If a login credential is equal to placement officer entered credentials
- 2: If matched then go to placement officer portal else display error message
- 3: end if
- 4: view details of students, company mails
- 5: send details of company to placed and not placed students
- 6: end if

//Pseudo code for Company

- 1: If a login credential is equal to placement officer entered credentials
- 2: If matched then go to company portal else display error message
- 3: end if
- 4: send the details of job description
- 5: end if

6.3.1 Implementation Code

Student Login Page

```
<?php

error_reporting();

session_start();

include_once 'connection.php';

if(isset($_SESSION['user'])=="")

{

    header("Location: homepage.html");

}

if(isset($_POST['submit']))

{

    $name=$_POST['name'];

    $user_id=$_POST['user_id'];

    $password=$_POST['password'];

    $res=mysqli_query($conn,"SELECT * FROM register WHERE user_id='$user_id' &&
name='$name'");

    $row=mysqli_fetch_array($res);

    if($row['password']==$password && $row['user_id']==$user_id)

    {
```

```

$_SESSION['user']=$row['user_id'];

echo "<script>alert('LOGIN');</script>";

$_SESSION['name']=$name;

header("Location: student.php");

}

else

{

    echo "<script>alert('Invalid Login Credentials');</script>";

}

}

mysqli_close($conn);

?>

```

Placement Login Page

```

<script>

function validation() {

    var user = document.forms["RegForm"]["user_id"];

    var password = document.forms["RegForm"]["password"];

    if(user.value == "") {

        window.alert("Please enter your user id.");

        user.focus();

        return false;

    }

    if(user.value != "878") {

        window.alert("Please enter valid user_id.");

        user.focus();

        return false;

    }

}

```



```

}

if(password.value == "") {

    window.alert("Please enter a password.");

    password.focus();

    return false;

}

if(password.value != "2019-20") {

    window.alert("Please enter valid password.");

    password.focus();

    return false;

}

else {

    window.location.href="placement1.php";

    return true;

}

}

</script>

```

Company Login Page

```

<?php

session_start();

include_once 'connection.php';

if(isset($_SESSION['user'])=="") {

    header("Location: homepage.html");

}

if(isset($_POST['submit'])) {

```

```

$name=$_POST['name'];

$user_id=$_POST['user_id'];

$password=$_POST['password'];

$res=mysqli_query($conn,"SELECT * FROM creg WHERE user_id='$user_id' &&
name='$name'");

$row=mysqli_fetch_array($res);

if($row['password']==$password && $row['user_id']==$user_id) {

    echo "<script>alert('LOGIN');</script>";

    $_SESSION['name']=$name;

    header("Location: c.php");

}

}

mysqli_close($conn);

?>

```

Student Details Page

```

<?php
include_once 'connection.php';
if(isset($_POST['btn-upload']))
{
    $fname=$_POST['fname'];
    $lname=$_POST['lname'];
    $roll=$_POST['roll'];
    $age=$_POST['age'];
    $gender=$_POST['gender'];
    $phone=$_POST['phone'];
    $cgpa=$_POST['cgpa'];
    $email=$_POST['email'];
    $file = rand(10000,100000)."-".$_FILES['file']['name'];
    $file_loc = $_FILES['file']['tmp_name'];
    $file_size = $_FILES['file']['size'];

```

```

$file_type = $_FILES['file']['type'];
$folder="uploads/";
move_uploaded_file($file_loc,$folder.$file);
$insertquery="INSERT INTO
resume(fname,lname,roll,age,gender,phone,cgpa,email,placed,file,type,size,username)
VALUES('$fname','$lname','$roll','$age','$gender','$phone','$cgpa','$email','$file','$file_type'
,$file_size)";
$res=mysqli_query($conn,$insertquery);
if($res) {
    echo "<script>alert('Uploaded Successfully');</script>";
}
}
?>

```

Company Mail Page

```

<?php
include_once('connection.php');
$sql="SELECT * FROM company1";
$result_set=mysqli_query($conn,$sql);
while($row=mysqli_fetch_array($result_set))
{
}
?>

```

Update Page

```

<?php
include 'connection.php';
if(isset($_POST['done'])) {
    $placed=$_POST['placed'];
    $id=$_GET['id'];
    $query=mysqli_query($conn,"UPDATE resume SET placed='$placed' WHERE id='$id'");
    header("location:placement1.php");
}

```

?>

Delete Page

```
<?php
include 'connection.php';
$id=$_GET['id'];
$q="DELETE FROM resume WHERE id='$id'";
mysqli_query($conn,$q);
header("location:placement1.php");
?>
```

Education Page

```
<?php
include 'connection.php';
if(isset($_POST['submit'])) {
    $course=$_POST['course'];
    $college=$_POST['college'];
    $grade=$_POST['grade'];
    $year=$_POST['year'];
    $insertquery="INSERT INTO education(course,college,grade,year)
        VALUES('$course','$college','$grade','$year')";
    $res=mysqli_query($conn,$insertquery);
    if($res) {
        echo "<script>alert('REQUIREMENTS SUBMITTED successfully');</script>";
    }
    else {
        echo "<script>alert('data not inserted properly')</script>";
    }
}
?>
```